LEVEL 1 GAME ENGINE - COMPLETE DETAILED EXPLANATION

1. LIBRARIES AND FUNCTION DECLARATIONS

The file begins with including stdlib.h and time.h.

stdlib.h provides access to the rand() function used for random enemy behavior.

time.h provides time() which is used to seed the random number generator.

Several iGraphics function prototypes are declared. These are external rendering functions such as iClear(), iShowImage(), iSetColor(), iFilledRectangle(), iFilledCircle(), iText(), and iLoadImage().

Declaring them tells the compiler that these functions exist in another file or library.

2. SCREEN AND BACKGROUND SYSTEM

sScreenW and sScreenH store the width and height of the game window.

sBg2Tex and sBgTex store texture IDs returned by iLoadImage(). These IDs represent loaded background images in GPU memory.

sCameraX tracks horizontal scrolling of the world.

sTransitionPoint1 and sTransitionPoint2 determine when the background switches.

This creates a side-scrolling effect.

3. SPRITE MANAGEMENT

FRAME_COUNT defines how many animation frames exist per action (6).

Arrays such as sKakashiRight[] store texture IDs for each animation frame.

There are separate arrays for standing right, standing left, walking right, walking left.

Enemies (Orochimaru) also have their own animation frame arrays.

Animation works by switching frame indices over time.

4. PHYSICS CONSTANTS

GRAVITY = -1.2 pulls the player downward each frame.

JUMP_FORCE = 18.0 sets initial upward velocity when jumping.

FRICTION = 0.82 gradually slows horizontal movement when no key is pressed.

MAX_HORIZONTAL_SPEED prevents infinite speed.

ACCELERATION controls how fast velocity increases when moving.

Together these values define the feel of movement.

5. GAME STRUCTURES

Enemy struct stores position (x,y), velocity, direction, health, alive state, and animation info.

Bullet struct stores position, velocity, active state, and radius.

EnemyBullet struct works similarly but damages the player.

These structs act as blueprints for game objects.

6. PLAYER STATE VARIABLES

sPlayerX and sPlayerY store player position.

sPlayerVelX and sPlayerVelY store velocity.

sIsOnGround prevents double jump.

sIsJumping tracks jump state.

sDirection determines which sprite to draw.

sPlayerHealth starts at 100.

These variables define the entire state of the player.

7. INITIALIZATION FUNCTION (initLevel1)

Stores screen size.

Loads all background and sprite images.

Seeds random generator using srand(time(0)).

Initializes enemies using resetEnemy().

Initializes bullets as inactive.

Resets player position and physics state.

This function prepares the level before gameplay begins.

8. DRAW FUNCTION (drawLevel1)

iClear() clears the previous frame.

Background is drawn based on camera position.

Player sprite is chosen depending on direction and movement.

Enemies are drawn only if alive.

Enemy health bars are drawn using red and green rectangles.

Player bullets are drawn as yellow circles.

Enemy bullets are drawn as violet circles.

Player health bar is drawn at the top left.

HUD text displays Level 1.

If sIsGameOver is true, a YOU DIED message is shown.

9. UPDATE FUNCTION (updateLevel1)

This function runs every frame and updates all game logic.

Checks for Game Over if player health <= 0.

Applies acceleration and friction for horizontal movement.

Applies gravity when player is airborne.

Updates position using velocity.

Checks ground collision and prevents falling below ground.

Handles sprite animation timing.

Moves enemies and makes them bounce at screen edges.

Enemies randomly shoot bullets toward player.

Player bullets check collision with enemies and reduce health.

Enemy bullets check collision with player and reduce health.

Inactive bullets are reused instead of created new.

This is the main game loop logic.

10. INPUT SYSTEM

Keyboard functions handle player input.

A/D or arrow keys move horizontally.

W/Space or Up arrow triggers jump.

S or Down arrow causes fast fall.

J key fires a bullet in facing direction.

KeyboardUp functions stop movement flags.

This system connects user input to velocity changes.

11. COMPLETE SYSTEM FLOW

Input -> Update Physics -> Collision Detection -> AI Logic -> Rendering -> Health Check -> Game Over.

This file effectively acts as a complete 2D side-scrolling engine.

It combines animation, physics, AI, rendering, and state management in one loop.

Overall architecture follows a standard game loop design pattern.